# 1. Introducción



Los **DTD** en realidad son una reminiscencia del lenguaje **SGML.** Los **XML Schema Definition** (o archivos **XSD**) son un desarrollo posterior del W3C (Consorcio World Wide Web).

En torno a 1999 se planteó una sintaxis que permitiera más capacidades de validación y que fuera más coherente con el lenguaje XML. Hoy en día es la arquitectura predominante para la validación, ya que se usa en **XQuery** y sobre todo en los servicios Web.

Es decir, este capítulo tratará el estudio de los **XML Schema Definition**, o archivos **XSD**.

**VENTAJAS**

- La sintaxis es XML, por lo que son analizables como cualquier XML.
- Permiten la validación de datos avanzados.

**DESVENTAJAS**

- Son más complejos de entender que los DTD.

A partir de ahora, se definirán los conceptos que rodean a los XSD de la manera más concisa posible. Después, sabes que estos conceptos serán trabajados a través de las hojas prácticas que os aporto en clase.

# 2. Estructura de los esquemas



Un esquema es un documento XML al que se le coloca la extensión **xsd**. Al ser un archivo XML tiene la estructura habitual de todo documento XML con la obligación de que el elemento raíz se llame **schema**.

**ETIQUETA SCHEMA**

La etiqueta **schema** identifica la raíz de un documento XML Schema. En esta etiqueta se declara el espacio de nombres estándar que utilizan los esquemas (y que permite diferenciar las etiquetas XML del esquema, respecto a las del documento XML), el cual se puede definir como el espacio de nombres por defecto, definir un prefijo **xs** para él (es la forma habitual) o bien definir un prefijo **xsd**. Es decir estas tres posibilidades:

- **<schema xmlns="**http://www.w3.org/2001/XMLSchema"**>**
- **<xs:schema xmlns:xs=**"http://www.w3.org/2001/XMLSchema"**>**
- **<xsd:schema xmlns:xsd=**"http://www.w3.org/2001/XMLSchema"**>**

A partir de ahí las etiquetas pertenecientes a XML Schema, usarán el prefijo indicado en su espacio de nombres (normalmente **xs**).

# 3. Asociar un XML a su esquema



Para que un documento XML siga las reglas definidas en un esquema, no disponemos de etiqueta **!DOCTYPE**; en su lugar utilizamos atributos especiales en el elemento raíz del documento XML.

Primero, (al igual que en el documento XMLSchema), necesitamos definir el espacio de nombres, el correspondiente al documento (que suele utilizar el prefijo **xs**, aunque se puede utilizar otro).Observa la declaración: **xmlns:xs="http://www.w3.org/2001/XMLSchema-instance".**

Además es necesario indicar dónde está el archivo XMLSchema que contiene las reglas de validación que se aplican al documento. Nosotros utilizaremos el atributo **xs:noNamespaceSchemaLocation**, con el que indicaremos que el documento se encuentra en un fichero local en vez de estar en un fichero en la Red (en cuyo caso debería utilizarse el atributo, **xs:schemaLocation=**http://...)

**Ejemplos completos de cómo validar un XML con tres esquemas diferentes:**

**Código XML**

```
<?xml version="1.0" encoding="UTF-8" ?>

<alumno xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
        xs:noNamespaceSchemaLocation="esquema.xsd">

    <nombre>Manuel</nombre>
    <apellidos>Ponte Alegre</apellidos>
</alumno>
```

**Esquema: esquema.xsd (formato muy legible)**

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="alumno">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="nombre" type="xs:string"/>
            <xs:element name="apellidos" type="xs:string"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>

</xs:schema>
```

**Otra versión más reutilizable**

```
<xs:element name="alumno" type="persona"/>

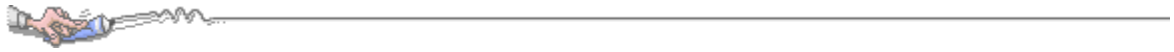
<xs:complexType name="persona">
    <xs:sequence>
        <xs:element name="nombre" type="xs:string"/>
        <xs:element name="apellidos" type="xs:string"/>
    </xs:sequence>
</xs:complexType>
```

# 3. Partes de un esquema



- **Elementos**, definidos con etiquetas **xs:element**. Para indicar los elementos permitidos en los documentos.
- **Atributos**, etiqueta **xs:attribute**.
- **Tipos simples**, que permiten definir los tipos simples de datos que podrá utilizar el documento XML. Lo hace la etiqueta **xs:simpleType**.
- **Tipos complejos**, mediante la etiqueta **xs:complexType**.
- **Documentación**, información utilizable por aplicaciones que manejen los esquemas. Etiquetas **xs:annotation**, **xs:documentation** y **xs:appInfo**.

# 4. Sintaxis de la definición de un elemento



En XML Schema la definición de un elemento XML se realiza mediante la etiqueta **element**. La sintaxis de los principales componentes es:

```
<xs:element
        name="nombre del elemento"
        type="tipo global de datos"
        minOccurs="número mínimo de veces"
        maxOccurs="máximo número de veces"
        default="valor por defecto"
        fixed="valor fijo"
/>
```

Al menos hay que indicar el nombre y el tipo. Por ejemplo, se puede definir un elemento como:

```
<?xml version="1.0" encoding="UTF-8"?>
 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
        <xs:element name="descripcion" type="xs:string" />
</xs:schema>
```

---

```
<?xml version="1.0" encoding="UTF-8" ?>

<descripcion xmlns:xs="http://www.w3.org/2001/XMLSchema-instance" xs:noNamespaceSchemaLocation="esquema.xsd">
        Información del único elemento que además es la raíz
</descripcion>
```

# 5. Tipos de datos



Para usarlos basta indicarlos en el atributo **type** de una etiqueta **element** o **attribute** (que son las que permiten crear elementos y atributos). De esta forma:

```
<xs:element name="descripcion" type="xs:string" />
```

Él código remarcado es el que indica que en los documentos XML basados en esta plantilla XMLSchema, habrá un elemento llamado "**descripción**" que contendrá datos de tipo **string**. El hecho de que se use el prefijo **xs** (es decir **xs:string**) es para indicar que es un tipo XML y por lo tanto usa el prefijo designado al espacio de nombres de XMLSchema.

Algunos de los tipos de datos más usuales son los siguientes:

**TEXTOS**

**string**. Representan textos con cualquier contenido excepto los símbolos <, > & y las comillas para los que se usará la entidad correspondiente.

**LÓGICOS**

**boolean.** Sólo puede contener los valores verdadero o falso, escritos como **true** o **false**, o como **1** (verdadero) ó **0** (falso).

**NÚMEROS**

**integer**. Permite utilizar números enteros.

**nonPositiveInteger**. Basado en **integer**. Representa números enteros que no son positivos (es decir cero y negativos).

**negativeInteger**. Basado en **integer**. Representa números enteros negativos (no vale el cero).

**nonNegativeInteger**. Basado en **integer**. Representa números enteros que no son negativos (es decir cero y positivos).

**positiveInteger**. Representa números enteros positivos puros (no vale el cero).

**float**. Permite utilizar números decimales en formato de coma flotante de precisión simple. El separador decimal es el punto. Ejemplos de valores: ***1, 2 3.4, -123,-124.76, 1E+3*** (significa 1000), ***1.45E+4*** (significa 14500), ***1.45E-4*** (significa 0,000145, es decir 1,45 · 10-4), ***INF*** (infinito), ***-INF*** (menos infinito).

**double**. Números decimales en formato de coma flotante de precisión doble. Es decir el mismo tipo de números pero con mejor precisión.

**decimal**. Representa números decimales en coma fija. Ocupan más internamente pero se representan de forma exacta.

**hexBinary**. Representa números binarios codificados en notación hexadecimal.

**FECHAS**

**duration**. Representa duraciones de tiempo en formato **ISO 8601 P**n**Y**n**M**n**DT**hms. Donde todos los signos **n** son números. Ejemplos de valores:

- **P1Y** significa Un año.
- **P1Y2M** significa un año y dos meses.
- **P1Y2M3D** significa un año y dos meses y tres días
- **P3D** significa tres días
- **P1Y2M3DT12H30M40.5S** significa un año, dos meses, tres días, doce horas treinta minutos y cuarenta segundosy medio
- **PT12H30M40.5S** doce horas treinta minutos y cuarenta segundosy medio

Como se observa la **P** es obligatoria y la **T** sirve para separar los valores de fecha de los valores hora.

**dateTime**. Representa fechas según el formato ISO 8601. El formato es yyyy-mm-ddThh:mm:ss , por ejemplo 1998-07-12T16:30:00.000 (12 de julio de 1998 a las 16:30). La T es obligatoria para separar la fecha de la hora.
**time**. Representa horas en el formato hh:mm:ss
**date**. Representa fecha en formato yyyy-mm-dd
**gYear**. Representa un año usando cuatro cifras.

# 5. Definir tipo de dato simple por restricción



Permiten establecer reglas complejas que deben de cumplir los datos. En este caso dentro de la etiqueta **simpleType** se indica una etiqueta **restriction**, dentro de la cual se establecen las posibles restricciones.

```
<xs:simpleType name="nombre">
        <xs:restriction base="tipo">
                ...definición de la restricción....
        <xs:restriction>
</xs:simpleType>
```

El atributo **base** sirve para indicar en qué tipo nos basamos al definir la restricción. El apartado restriction puede tener numerosas etiquetas que permiten establecer las restricciones.

Las posibles restricciones que se pueden establecer son:

**TAMAÑOS DE TEXTO**. Indica tamaños máximos y mínimos que debe de tener el texto.

- **minLength**. Indica el mínimo número de caracteres. Eso lo hace mediante el atributo **value,** en el que se indica el número mínimo.
- **maxLength**. Indica un tamaño máximo de caracteres o de dígitos numéricos. Usa el mismo atributo **value**.
- **length**. Indica un tamaño fijo de caracteres para el tipo. Es decir si indicados **length** con **value="9"** el texto deberá tener exactamente nueve caracteres.

```
<?xml version="1.0" encoding="UTF-8" ?>
<alumno xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
          xs:noNamespaceSchemaLocation="esquema.xsd">

   <nombre>Ana</nombre>
    <apellidos>Tomía Perfecta</apellidos>
</alumno>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

        <xs:element name="alumno">
                <xs:complexType>
                        <xs:sequence>
                                <xs:element name="nombre">
                                        <xs:simpleType>
                                                <xs:restriction base="xs:string">
                                                <xs:minLength value="3"/>
                                                <xs:maxLength value="15"/>
                                                </xs:restriction>
                                        </xs:simpleType>
                                </xs:element>
                                <xs:element name="apellidos" type="xs:string"/>
                        </xs:sequence>
                </xs:complexType>
        </xs:element>
</xs:schema>
```

- Ahora, trata de modificar el código anterior para que la restricción también afecte al apellido. Pero con un código reutilizable.

**DÍGITOS MÁXIMOS**. Parecido al anterior pero trabajando con números. Indica las posibles cifras que puede tener el número.

- **totalDigits**. Número máximo de dígitos del número, incluyendo los decimales. El atributo value indica el número máximo deseado
- **fractionDigits**. Máximo número de decimales que puede tener el número.

```
<?xml version="1.0" encoding="UTF-8" ?>
<empleado xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
          xs:noNamespaceSchemaLocation="esquema.xsd">

  <nombre>Helen</nombre>
  <apellidos>Chufe</apellidos>
  <salario>1125.75</salario>

</empleado>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

        <xs:element name="empleado">
                <xs:complexType>
                        <xs:sequence>
                                <xs:element name="nombre" type="xs:string"/>
                                <xs:element name="apellidos" type="xs:string"/>
                                <xs:element name="salario">
                                        <xs:simpleType>
                                                <xs:restriction base="xs:decimal">
                                                        <xs:totalDigits value="6" />
                                                        <xs:fractionDigits value="2" />
                                                </xs:restriction>
                                        </xs:simpleType>
                                </xs:element>
                        </xs:sequence>
                </xs:complexType>
        </xs:element>
</xs:schema>
```

**MÁXIMOS Y MÍNIMOS NUMÉRICOS**. Restringe valores numéricos asignando topes a los mismos. Sirve para números y para valores de tiempo o duración.

- **minExclusive**. Establece un valor mínimo. El valor debe ser mayor que el establecido por la etiqueta a través del atributo **value**.
- **maxExclusive**. Establece un valor máximo. El valor debe ser menor que el establecido por la etiqueta a través del atributo **value**.
- **minInclusive**. Establece un valor mínimo. El valor debe ser mayor o igual que el establecido por la etiqueta a través del atributo **value**.
- **maxInclusive**. Establece un valor máximo. El valor debe ser menor o igual que el establecido por la etiqueta a través del atributo **value**.

```
<xs:element name="salario">
        <xs:simpleType>
                <xs:restriction base="xs:decimal">
                        <xs:totalDigits value="6" />
                        <xs:fractionDigits value="2" />
                        <xs:minInclusive value="500" />
                        <xs:maxInclusive value="3000" />
                </xs:restriction>
        </xs:simpleType>
</xs:element>
```

- Modifica el XML anterior y su XSD añadiendo estas dos nuevas restricciones.

**ENUMERACIONES**. Limitan el contenido a una lista de valores. Se realizan con una sucesión de etiquetas **enumeration**. Estas etiquetas se pueden combinar con las propiedades anteriores.

Ejemplo de enumeración:

```
<xs:simpleType name="tipo_sexo">
        <xs:restriction base="xs:string">
                <xs:enumeration value="Hombre"/>
                <xs:enumeration value="Mujer"/>
        </xs:restriction>
</xs:simpleType>
```



Añade al código con el que estamos trabajando, esta nueva restricción de sexo.

**UNA PISTA.** En el XSD, además del código anterior, deberás añadir un nuevo elemento en la secuencia:

```
            <xs:element name="sexo" type="tipo_sexo"/>
        </xs:sequence>
```

**PLANTILLAS**. Permite establecer **expresiones regulares;** es decir, un texto con símbolos especiales que permiten establecer expresiones que ha de cumplir el contenido. Las expresiones regulares son bien conocidas por casi todos los programadores y dan una potencia increíble para establecer restricciones de texto avanzadas, lo que las hace muy utilizadas.

Las plantillas se manejan con etiquetas **pattern** a las que, en el atributo **value**, se indica un texto con símbolos especiales que especifica la expresión a cumplir. Los símbolos que se pueden utilizar son:

| Símbolos | Significado |
|---|---|
| **.** | Cualquier carácter. |
| **\d** | Cualquier dígito del 0 al 9. |
| **\D** | Cualquier carácter que no sea un dígito del 0 al 9. |
| **x*** | x puede aparecer cero o más veces. |
| **x+** | x debe aparecer al menos una vez. |
| **x?** | x puede aparecer una vez o no aparecer. |
| **[abc] o [a\|b\|c]** | Cualquier carácter indicado entre los corchetes: a, b o c. |
| **[a-z]** | Cualquier carácter de la a la z. |
| **x{n}** | x debe aparecer n veces. |
| **x{n,m}** | x debe aparecer entre n y m veces. |
| **x{n,}** | x debe aparecer al menos n veces. |
| **Cualquier texto** | Hace que sólo se pueda escribir ese texto. |

Ejemplo de **pattern**:

```
<xs:simpleType name="tipo_dni">
        <xs:restriction base="xs:string">
                <xs:pattern value="[0+-9]{8}[A-Z]"/>
        </xs:restriction>
</xs:simpleType>
```

- Prueba en el XML y XSD el código anterior. Como puedes imaginar, ahora debes añadir al XML el elemento dni. Después, comprueba que valores distintos a los definidos en la plantilla anterior, hacen que el documento no sea válido.

# 6. Atributos

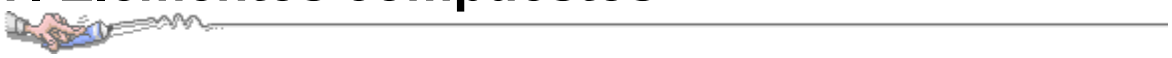


Los atributos se definen de forma parecida a los elementos. Su sintaxis de definición es:

```
<xs:attribute
        name="nombre del elemento"
        type="tipo global de datos"
        use="uso" <!-- prohibited, optional o required
>
```

Los **atributos** sólo pueden asignarse a los elementos mediante **tipos complejos** de datos. El uso hace referencia a si al atributo no se le puede asignar ningún valor en el documento XML (valor **prohibited**), si su uso es opcional (valor **optional**) o si es **obligatorio** darle valor.

# 7. Elementos compuestos



Hasta ahora sólo hemos trabajado con contenidos simples, **simpleType** que contienen sólo información en el interior. O bien para indicar posibles valores a los atributos

**Los tipos compuestos** permiten definir contenidos más complejos. Puesto que lo normal es que los elementos de un documento XML puedan contener otros elementos y por supuesto atributos, es lógico que la mayoría de elementos indiquen mediante tipos compuestos su contenido.

Los tipos compuestos se definen con la etiqueta **complexType**. En esa etiqueta podemos utilizar diferentes modelos de definición de contenidos.

En realidad los elementos XML desde el punto de vista de la sintaxis XML Schema pueden tener cuatro tipos de contenido:

- **Vacíos (Empty)**. No pueden contener ni más elementos dentro ni texto.
- **Contenido simple** (**Simple Content**). Sólo admite texto y no otros elementos.
- **Contenido compuesto** (**Complex Content**). Pueden contener otros elementos.
- **Contenido mixto (Mixed Content)**. Pueden contener tanto texto como más elementos.

## ELEMENTOS VACÍOS

Para indicar que un elemento es vacío basta con no indicar valores e indicar el nombre del elemento sin indicar tipo de datos alguno. Analiza el siguiente código

```
<xs:element name="estado">
    <xs:complexType>
        <xs:attribute name="casado" type="xs:boolean"/>
    </xs:complexType>
</xs:element>
```

En este caso el elemento **estado** sólo dispone de un **atributo** llamado **casado**, no será posible meter ningún contenido en la etiqueta de casado. Si, además, no deseamos atributos entonces simplemente no habrá etiquetas **attribute**.



De nuevo, añade el código anterior al XSD, modifica el XML para que valide y, después, trata de hacer las pruebas que consideres con el atributo del elemento: true, false, 1, 0 y con otros valores.

## DEFINICIÓN DE CONTENIDOS CON TEXTO

Se trata de usar la etiqueta **<xs:element** al estilo que se ha usado en los ejemplos. Es decir, se indica el nombre y el tipo de datos e incluso se pueden indicar atributos dentro del apartado **complexType**. Tal y como se ha hecho en el apartado anterior.

```
<xs:element name="nombre" type="xs:string"/>
```

## TIPO COMPLEJO A PARTIR DE TIPO SIMPLE

En este caso se indica que el elemento posee contenido simple en la etiqueta **complexType** dentro de la cual se indican los atributos, mientras que es la etiqueta del contenido simple (**simpleContent**) la que poseerá el tipo de datos para el contenido del elemento.

Los atributos se deben indicar en el apartado **extension** que es el encargado de indicar los tipos.

```
<xs:element name="nacionalidad">
    <xs:complexType>
        <xs:simpleContent>
            <xs:extension base="xs:string">
                <xs:attribute name="comunitario" type="xs:boolean"/>
            </xs:extension>
        </xs:simpleContent>
    </xs:complexType>
</xs:element>
```

En el ejemplo, **nacionalidad** es un elemento de tipo **boolean** (lógico) que contiene un **atributo** llamado **comunitario** (string). **Es enrevesado, pero hay que hacerlo así**.

## DEFINICIÓN DE CONTENIDOS COMPUESTOS POR SECUENCIAS

Como se ha comentado antes, los contenidos compuestos se refieren a los elementos que contienen otros elementos (pero nunca texto libre). Hay tres posibles tipos de elementos a contener: **secuencias**, **elecciones** y contenidos libres (**all**). Además se pueden incorporar atributos.

```
<?xml version="1.0" encoding="UTF-8" ?>
<pelicula xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
            xs:noNamespaceSchemaLocation="esquema.xsd">

        <titulo>Piratas del Caribe:La Maldición de la Perla Negra</titulo>
        <datos>
                <director>Gore Verbinski</director>
                <estreno>2003</estreno>
                <actor>Johnny Depp</actor>
                <actor>Keira Knightley</actor>
                <actor>Orlando Bloom</actor>
        </datos>
</pelicula>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="pelicula">
        <xs:complexType>
                <xs:sequence>
                        <xs:element name="titulo" type="xs:string"/>
                        <xs:element name="datos" type="datos_peli"/>
                </xs:sequence>
        </xs:complexType>
</xs:element>

<xs:complexType name="datos_peli">
    <xs:sequence>
        <xs:element name="director" type="xs:string"/>
        <xs:element name="estreno" type="xs:decimal"/>
        <xs:element name="actor" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>

</xs:schema>
```

La etiqueta **sequence** posee los atributos **minOccurs** y **maxOccurs** para indicar que el bloque de la secuencia se puede repetir. En el ejemplo, actor puede aparecer una vez (como mínimo) o varias veces (unbounded)

## DEFINICIÓN DE CONTENIDOS COMPUESTOS POR ELECCIÓN

Sirven para permitir elegir uno de entre varios elementos. Su funcionamiento es el mismo que en las secuencias, pero en este caso se utiliza una etiqueta llamada **choice**.

En el ejemplo, el elemento **identificación**, consta de dos posibles elementos **firma** y **código** de los que sólo se podrá incluir uno.

La etiqueta **choice** también puede utlilizar los atributos **minOccurs** y **maxOccurs**.

```
<?xml version="1.0" encoding="UTF-8" ?>
<pelicula xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
            xs:noNamespaceSchemaLocation="esquema.xsd">

        <titulo>Piratas del Caribe:La Maldición de la Perla Negra</titulo>
        <datos>
        <director>Gore Verbinski</director>
        <estreno>2003</estreno>
        <actor>Johnny Depp</actor>
        <actor>Keira Knightley</actor>
        <actor>Orlando Bloom</actor>
        <critica>
                <desfavorable>Muy entretenida y con mucha acción</desfavorable>
        </critica>
        </datos>
</pelicula>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="pelicula">
        <xs:complexType>
                <xs:sequence>
                        <xs:element name="titulo" type="xs:string"/>
                        <xs:element name="datos" type="datos_peli"/>
                </xs:sequence>
        </xs:complexType>
</xs:element>

<xs:complexType name="datos_peli">
  <xs:sequence>
    <xs:element name="director" type="xs:string"/>
    <xs:element name="estreno" type="xs:decimal"/>
    <xs:element name="actor" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
    <xs:element name="critica">
          <xs:complexType>
            <xs:choice>
              <xs:element name="favorable" type="xs:string"/>
              <xs:element name="desfavorable" type="xs:string"/>
            </xs:choice>
          </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>

</xs:schema>
```

## DEFINICIÓN DE CONTENIDOS COMPUESTOS CON LA ETIQUETA "all"

Se trata de una posibilidad similar a **choice** y **sequence** que se utiliza de la misma forma y que tiene como diferencia principal que los elementos que contiene pueden aparecer en el **orden que quieran**.

```
<?xml version="1.0" encoding="UTF-8" ?>
<pelicula xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
        xs:noNamespaceSchemaLocation="esquema.xsd">

    <datos>
    <director>Gore Verbinski</director>
    <estreno>2003</estreno>
    <actor>Johnny Depp</actor>
    <actor>Keira Knightley</actor>
    <actor>Orlando Bloom</actor>
    <critica>
        <desfavorable>Muy entretenida y con mucha acción</desfavorable>
    </critica>
    </datos>
    <titulo>Piratas del Caribe:La Maldición de la Perla Negra</titulo>
</pelicula>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="pelicula">
    <xs:complexType>
        <xs:all>
            <xs:element name="titulo" type="xs:string"/>
            <xs:element name="datos" type="datos_peli"/>
        </xs:all>
    </xs:complexType>
</xs:element>

<xs:complexType name="datos_peli">
  <xs:sequence>
    <xs:element name="director" type="xs:string"/>
    <xs:element name="estreno" type="xs:decimal"/>
    <xs:element name="actor" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
    <xs:element name="critica">
        <xs:complexType>
          <xs:choice>
            <xs:element name="favorable" type="xs:string"/>
            <xs:element name="desfavorable" type="xs:string"/>
          </xs:choice>
        </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>

</xs:schema>
```

# 8. Añadiendo atributos



En los apartados **complexType**, los atributos del elemento se definen al final del apartado **complexType**, justo antes de cerrarlo. Ejemplo:

**PRIMERA VERSIÓN**

```
<?xml version="1.0" encoding="UTF-8" ?>
<alumnos  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
                xs:noNamespaceSchemaLocation="esquema.xsd">

        <alumno sexo="Hombre">
                <nombre>Igor</nombre>
                <apellidos>Dito Delgado</apellidos>
        </alumno>

        <alumno sexo="Mujer">
                <nombre>Ana</nombre>
                <apellidos>Tomía Perfecta</apellidos>
        </alumno>

</alumnos>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="alumnos" >
  <xs:complexType >
    <xs:sequence>
      <xs:element name="alumno" type="datos_alumno" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>


<xs:complexType name="datos_alumno">
   <xs:sequence>
      <xs:element name="nombre" type="xs:string"/>
      <xs:element name="apellidos" type="xs:string" />
   </xs:sequence>
   <xs:attribute name="sexo" type="xs:string"/>
</xs:complexType>

</xs:schema>
```



Prueba qué ocurre si en el XML eliminas un atributo Te darás cuenta que sigue siendo válido. Por defecto, la declaración de uso es opcional. Después, añade el atributo **use=”required”** y comprueba que **“sexo”** se convierte en obligatorio.

**SEGUNDA VERSIÓN.** Observa ahora que se ha añadido un segundo atributo. Uno es opcional pero el otro es obligatirio (**required**).

```
<?xml version="1.0" encoding="UTF-8" ?>
<alumnos  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
                xs:noNamespaceSchemaLocation="esquema.xsd">

        <alumno sexo="Hombre" nacimiento="2000-12-28">
                <nombre>Igor</nombre>
                <apellidos>Dito Delgado</apellidos>
        </alumno>

        <alumno sexo="Mujer">
                <nombre>Ana</nombre>
                <apellidos>Tomía Perfecta</apellidos>
        </alumno>

</alumnos>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="alumnos" >
  <xs:complexType >
    <xs:sequence>
      <xs:element name="alumno" type="datos_alumno" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>


<xs:complexType name="datos_alumno">
            <xs:sequence>
                        <xs:element name="nombre" type="xs:string"/>
                        <xs:element name="apellidos" type="xs:string" />
            </xs:sequence>

            <xs:attribute name="sexo" type="xs:string" use="required"/>
            <xs:attribute name="nacimiento" type="xs:date" />
</xs:complexType>

</xs:schema>
```

**TERCERA VERSIÓN.** En este código encontrarás explicación a algo que vimos en la página nueve: **tipo complejo a partir de un tipo simple**.

```
<?xml version="1.0" encoding="UTF-8" ?>
<alumnos  xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
                xs:noNamespaceSchemaLocation="esquema.xsd">

        <alumno sexo="Hombre" nacimiento="2000-12-28">
                <nombre>Igor</nombre>
                <apellidos>Dito Delgado</apellidos>
        </alumno>

        <alumno sexo="Mujer">
                <nombre>Ana</nombre>
                <apellidos>Tomía Perfecta</apellidos>
        </alumno>

</alumnos>
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="alumnos" >
  <xs:complexType >
    <xs:sequence>
      <xs:element name="alumno" type="datos_alumno" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>


<xs:complexType name="datos_alumno">
        <xs:sequence>
                <xs:element name="nombre" type="xs:string"/>
                <xs:element name="apellidos" type="xs:string" />
        </xs:sequence>

        <xs:attribute name="sexo" type="xs:string" use="required"/>
        <xs:attribute name="nacimiento" type="xs:date" />
</xs:complexType>

</xs:schema>
```



Y para aprovechar lo que queda de hoja, **¿podrías intuir qué haría el siguiente código en el esquema?** Lo probaremos en las hojas de prácticas.

```
<xs:attribute name="sexo">
        <xs:simpleType>
                <xs:restriction base="xs:string">
                        <xs:enumeration value="Hombre" />
                        <xs:enumeration value="Mujer" />
                </xs:restriction>
        </xs:simpleType>
</xs:attribute>
```

# 9. Anexo sobre plantillas (pattern)



Unos ejercicios y sus soluciones que encontré por Internet:

Ejercicios

| | |
|---|---|
| 1) | "Capítulo 0", "Capítulo 1", "Capítulo 2"... "Capítulo 9". (Solo se permite un dígito). |
| 2) | "Capítulo 0", "Capítulo 1", "Capítulo 2"... "Capítulo 99". (Uno o dos dígitos). |
| 3) | "Capítulo 1", "Capítulo 2", "Capítulo 3"... "Capítulo 99". (No se permite "Capítulo 0"). |
| 4) | "Capítulo 0", "Capítulo 1", "Capítulo 2"... "Capítulo 99"... "Capítulo 100"... (Uno o más dígitos). |
| 5) | Cualquier valor de dos caracteres, cuyo primer carácter sea distinto de un dígito (0-9) y cuyo segundo carácter sea "Z": "aZ"... "zZ", "AZ"... "ZZ", "?Z", "=Z", "*Z"... |
| 6) | "ABBC", "ABBBC", "ABBBBC", "ABBBBBC". |
| 7) | El primer carácter debe ser "R". A continuación, deben aparecer obligatoriamente dos o más "S". Finalmente, puede aparecer o no, un dígito del 3 al 8: "RSS", "RSSS"... "RSS3"... "RSS8", "RSSS3"... "RSSS8"... "RSSSSSSSSSS7"... |
| 8) | Cualquier valor que contenga en primer lugar "COD", después tres dígitos (0-9) y, finalmente, uno o más caracteres cualesquiera: "COD645pera", "COD646manzana"... |

**Solución:**

| | |
|---|---|
| 1) | Capítulo \d<br>Capítulo [0-9]<br>Capítulo [0\|1\|2\|3\|4\|5\|6\|7\|8\|9] |
| 2) | Capítulo \d\d?<br>Capítulo \d{1,2}<br>Capítulo [0-9][0-9]?<br>Capítulo [0\|1\|2\|3\|4\|5\|6\|7\|8\|9][0\|1\|2\|3\|4\|5\|6\|7\|8\|9]? |
| 3) | Capítulo [1-9]\d?<br>Capítulo [1-9][0-9]?<br>Capítulo [1\|2\|3\|4\|5\|6\|7\|8\|9][0\|1\|2\|3\|4\|5\|6\|7\|8\|9]? |
| 4) | Capítulo \d+<br>Capítulo \d\d*<br>Capítulo [0-9]+<br>Capítulo [0\|1\|2\|3\|4\|5\|6\|7\|8\|9]+ |
| 5) | \DZ<br>\D[Z] |
| 6) | AB{2,5}C<br>ABBB?B?B?C |
| 7) | RSS+[3-8]?<br>RSSS*[3-8]?<br>RS{2,}[3-8]?<br>RSS+[3\|4\|5\|6\|7\|8]? |
| 8) | COD\d{3}.+<br>COD\d\d\d.+<br>COD[0-9][0-9][0-9].+ |